

App Domain in .NET Framework

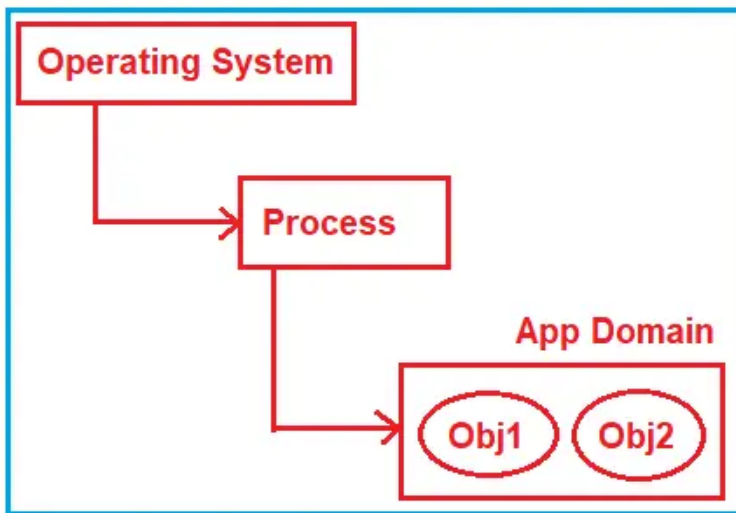
In this article, I am going to discuss **App Domain in .NET Framework** and in what scenarios we need them with examples. Please read our previous article where we discussed [Assembly, DLL, and EXE](#) in detail. The App Domain (Application Domain) in .NET Framework is a logically isolated container inside which the .NET Code runs. At the end of this article, you will understand what is App Domain and how to Create a Custom App Domain in .NET Framework with Examples.

Understanding App Domain in .NET Framework:

Let us understand App Domain in .NET Framework with an Example. Please create a Console Application using C# language and then copy and paste the following code into the **Program.cs** class file. This is a very simple application. Here we created two classes **MyClass1** and **MyClass2**. Then we create objects of both these classes inside the Main method of the Program class and load these two classes into the console application.

```
using System;
namespace AppDomainDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            MyClass1 obj1 = new MyClass1();
            MyClass2 obj2 = new MyClass2();
            Console.Read();
        }
    }
    public class MyClass1
    {
    }
    public class MyClass2
    {
    }
}
```

Now when you run the above application or EXE, what will happen internally let us discuss. Here, the EXE runs as a Process inside the Operating System. Inside the Process, we have one App Domain by default loaded, and inside that App Domain, both the objects (obj1 and obj2) are running as shown in the below image.



Note: By default always there is an App Domain under which our .NET Code runs.

Need for App Domain in .NET Application:

Let us understand the need for App Domain in .NET Applications. Suppose you want to consume a Third Party DLL. That DLL you may get from the Internet or from any other third parties. Here, you have one doubt i.e. the Third Party DLL access your **D:\ Drive**. Suppose you want to use the Third DLL which you download from the internet for reporting purposes, but there is some kind of virus that creates a file in your C:/ Drive instead of working as a reporting tool.

Here, we are not downloading any DLL from the internet, instead, we will create a class as shown below which will act as the Third Party DLL.

```
[Serializable]
public class ThirdParty
{
    public ThirdParty()
    {
        Console.WriteLine("Third Party DLL Loaded");
        System.IO.File.Create(@"D:\xyz.txt");
    }

    ~ThirdParty()
    {
        Console.WriteLine("Third Party DLL Unloaded");
    }
}
```

Now, if you simply use the ThirdParty class with the default App Domain, then it can have access to your D:\ Drive. Let's modify the Program.cs class file as shown below.

```

using System;
namespace AppDomainDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Third Party DLL
            ThirdParty Obj3 = new ThirdParty();
            //Own DLL
            MyClass1 obj1 = new MyClass1();
            MyClass2 obj2 = new MyClass2();
            Console.Read();
        }
    }
    [Serializable]
    public class ThirdParty
    {
        public ThirdParty()
        {
            Console.WriteLine("Third Party DLL Loaded");
            System.IO.File.Create(@"D:\xyz.txt");
        }
        ~ThirdParty()
        {
            Console.WriteLine("Third Party DLL Unloaded");
        }
    }
    public class MyClass1
    {
    }
    public class MyClass2
    {
    }
}

```

Now, when you execute the above code it will create the text file in the D Drive. But we want to restrict the third-party DLL to access our D drive. We can do this by creating a separate App Domain for the Third Party DLL and then we will provide settings to that App Domain so that, it will not access our D Drive.

How to Create a Custom App Domain in .NET Framework?

Let us see how to create our own App Domain in .NET Framework and also see how we will run the Third Party DLL inside that Custom App Domain. Then we will see how to provide permission to restrict access to the D Drive. Please have a look at the following code which shows how to create a custom app domain using C# language. The code is self-explain, please go through the comment line.

```

//Step1:
//Create custom App Domain
//CreateDomain: Creates a new application domain with the specified name.
AppDomain customDomain = AppDomain.CreateDomain("customDomain");

//Step2:
//Get the Type of ThirdParty using the typeof method by passing the ThirdParty class name
Type thirdParty = typeof(ThirdParty);

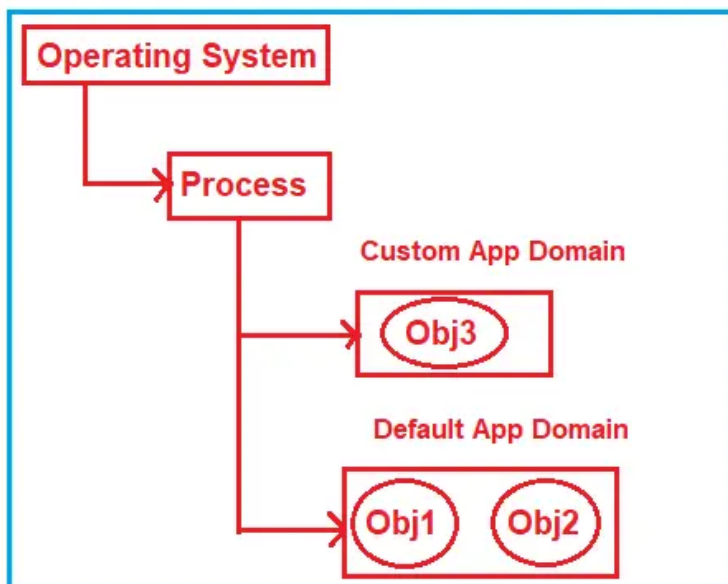
//Step3:
//Create an object of ThirdParty using the customDomain i.e. load the ThirdParty
//To Create an Object, we need to call the CreateInstanceAndUnwrap method of customDomain object
customDomain.CreateInstanceAndUnwrap(
    //Gets the display name of the assembly.
    thirdParty.Assembly.FullName,

    //Gets the fully qualified name of the type, including its namespace
    //but not its assembly.
    thirdParty.FullName);

//Step4:
//Unload the Custom App Domain
AppDomain.Unload(customDomain);

```

Once you understood how to create a Custom App Domain in C#. Let us see what we want to do now. We want to execute the Third Party DLLs using the Custom App Domain, while our own classes we want to execute inside the default app domain which is shown in the below image.



The complete code to implement the above requirement is given below. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace AppDomainDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Step1:
        //Create custom App Domain
        //CreateDomain: Creates a new application domain with the specific
        AppDomain customDomain = AppDomain.CreateDomain("customDomain");
        //Step2:
        //Get the Type of ThirdParty using the typeof method by passing th
ThirdParty class name
        Type thirdParty = typeof(ThirdParty);
        //Step3:
        //Create an object of ThirdParty using the customDomain i.e. load
ThirdParty
        //To Create an Object, we need to call the CreateInstanceAndUnwrap
customDomain object
        customDomain.CreateInstanceAndUnwrap(
                                //Gets the display name of the assembly.
                                thirdParty.Assembly.FullName,
                                //Gets the fully qualified name of the type,
its namespace
                                //but not its assembly.
                                thirdParty.FullName);

        //Step4:
        //Unload the Custom App Domain
        AppDomain.Unload(customDomain);
        //Own DLL
        MyClass1 obj1 = new MyClass1();
        MyClass2 obj2 = new MyClass2();
        Console.Read();
    }
}
[Serializable]
public class ThirdParty
{
    public ThirdParty()
    {
        Console.WriteLine("Third Party DLL Loaded");
        System.IO.File.Create(@"D:\xyz.txt");
    }
    ~ThirdParty()
    {
        Console.WriteLine("Third Party DLL Unloaded");
    }
}
public class MyClass1
{
}
public class MyClass2
{
}

```

```
}
```

Now if you execute the above code, then you will see that, it will also create the text file in the D Drive. This is because we have run the Third Party DLL using a custom app domain but till now we have not written any logic to restrict access to the D Drive.

How to Restrict Access to a Custom App Domain to D Drive in C#?

Let us see how to restrict the Custom Application Domain to access our D Drive. In order to restrict the Custom Application Domain to access D Drive, we need to create a permission object and restrict No Access to D Drive and then create a setup for the App Domain. And finally, we need to use both permissions and setup while creating the Custom Application Domain. The complete code is given below and the code is self-explain so, please go through the comment lines for a better understanding.

```
using System;
using System.Security;
using System.Security.Permissions;
namespace AppDomainDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Create Permission object
            var permission = new PermissionSet(PermissionState.None);
            //Permission for the code to run.
            //Execution: Without this permission, Managed Code will not be Executed
            permission.AddPermission(
                new SecurityPermission(SecurityPermissionFlag.Execution)
            );
            //Set No Access to C Drive,
            //NoAccess: No access to a file or directory.
            permission.AddPermission(
                new FileIOPermission(FileIOPermissionAccess.NoAccess, @"D:\")
            );
            //Step2: Create setup for App Domain
            var setUp = new AppDomainSetup
            {
                //CurrentDomain: Gets the current application domain
                //SetupInformation: Gets the application domain configuration
                //ApplicationBase: Gets or sets the name of the directory containing the application.
                ApplicationBase =
                AppDomain.CurrentDomain.SetupInformation.ApplicationBase
            };
            //Step3: Create custom App Domain
            //Create custom App Domain using the setup and permission
            //CreateDomain: Creates a new application domain with the specified name and permission
            AppDomain customDomain = AppDomain.CreateDomain("customDomain", null, permission);
        }
    }
}
```

```

        //Step4:
        //Get the Type of ThirdParty using the typeof method by passing th
ThirdParty class name
        Type thirdParty = typeof(ThirdParty);
        //Step5:
        //Create an object of ThirdParty using the customDomain i.e. load
ThirdParty
        //To Create an Object, we need to call the CreateInstanceAndUnwrap
customDomain object
        customDomain.CreateInstanceAndUnwrap(
                                //Gets the display name of the assembly.
                                thirdParty.Assembly.FullName,
                                //Gets the fully qualified name of the type,
its namespace
                                //but not its assembly.
                                thirdParty.FullName);

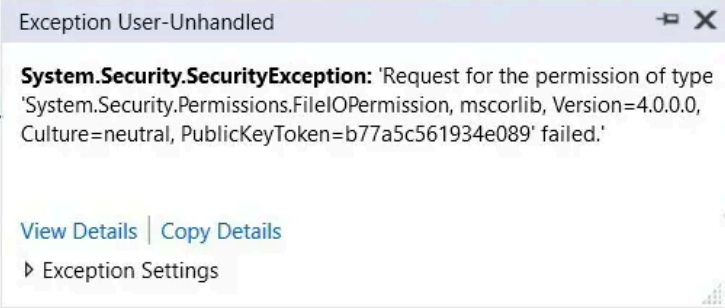
        //Step6:
        //Unload the Custom App Domain
        AppDomain.Unload(customDomain);
        //Own DLL
        MyClass1 obj1 = new MyClass1();
        MyClass2 obj2 = new MyClass2();
        Console.Read();
    }
}
[Serializable]
public class ThirdParty
{
    public ThirdParty()
    {
        Console.WriteLine("Third Party DLL Loaded");
        System.IO.File.Create(@"D:\xyz.txt");
    }
    ~ThirdParty()
    {
        Console.WriteLine("Third Party DLL Unloaded");
    }
}
public class MyClass1
{
}
public class MyClass2
{
}
}

```

Now when you execute the above code, the line from which it will try to access and create a file in the D Drive will through the following exception.

```
[Serializable]
public class ThirdParty
{
    public ThirdParty()
    {
        Console.WriteLine("Third Party DLL Loaded");
        System.IO.File.Create(@"D:\xyz.txt");
    }
}

~ThirdParty()
{
    Console.WriteLine("Third Party DLL Unloaded");
}
```



But irrespective of the exception in the Custom App Domain, if you want to execute the Default App Domain, then you need to put the logic of the Custom App Domain inside the try-catch block as shown in the below code.

```
using System;
using System.Security;
using System.Security.Permissions;
namespace AppDomainDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Create Permission object
            var permission = new PermissionSet(PermissionState.None);
            //Permission for the code to run.
            //Execution: Without this permission, Managed Code will not be Executed
            permission.AddPermission(
                new SecurityPermission(SecurityPermissionFlag.Execution)
            );
            //Set No Access to C Drive,
            //NoAccess: No access to a file or directory.
            permission.AddPermission(
                new FileIOPermission(FileIOPermissionAccess.NoAccess, @"D:\")
            );
            //Step2: Create setup for App Domain
            var setUp = new AppDomainSetup
            {
                //CurrentDomain: Gets the current application domain
                //SetupInformation: Gets the application domain configuration
            }
        }
    }
}
```

```

        //ApplicationBase: Gets or sets the name of the directory cont
application.
        ApplicationBase =
AppDomain.CurrentDomain.SetupInformation.ApplicationBase
    };
    //Step3: Create custom App Domain
    //Create custom App Domain using the setup and permission
    //CreateDomain: Creates a new application domain with the specific
AppDomain customDomain = AppDomain.CreateDomain("customDomain", null,
permission);
    try
    {
        //Step4:
        //Get the Type of ThirdParty using the typeof method by passing
ThirdParty class name
        Type thirdParty = typeof(ThirdParty);
        //Step5:
        //Create an object of ThirdParty using the customDomain i.e. [
ThirdParty
        //To Create an Object, we need to call the CreateInstanceAndUnwrap
of customDomain object
        customDomain.CreateInstanceAndUnwrap(
            //Gets the display name of the assembly.
            thirdParty.Assembly.FullName,
            //Gets the fully qualified name of the type
including its namespace
            //but not its assembly.
            thirdParty.FullName);
    }
    catch (Exception Ex)
    {
        Console.WriteLine($"Exception Occurred: {Ex.Message}");
        //Step6:
        //Unload the Custom App Domain
        AppDomain.Unload(customDomain);
    }
    //Own DLL
    MyClass1 obj1 = new MyClass1();
    MyClass2 obj2 = new MyClass2();
    Console.Read();
}
}
[Serializable]
public class ThirdParty
{
    public ThirdParty()
    {
        Console.WriteLine("Third Party DLL Loaded");
        System.IO.File.Create(@"D:\xyz.txt");
    }
    ~ThirdParty()

```

```

        {
            Console.WriteLine("Third Party DLL Unloaded");
        }
    }
    public class MyClass1
    {
    }
    public class MyClass2
    {
    }
}

```

Output:

```

Third Party DLL Loaded
Exception Occurred: Exception has been thrown by the target of an invocation.
Third Party DLL Unloaded

```

Advantages of using App Domain in .NET Application:

The App Domain (Application Domain) is a logically isolated container inside a process. In this logical isolation, you can load and run .NET Code in an isolated manner. The following are the advantages of using the App Domain.

1. You can load and unload DLL inside these logical containers without one container affecting the other. So, if there are issues in one application domain you can unload that application domain, and the other application domain work without issues.
2. If you have a Third Party DLL and for some reason, you don't trust the third-party code. You can run that DLL in an isolated app domain with fewer privileges. For example, you can say that the DLL cannot access your "D:\:" drive. And other DLLs that you trust you can run with full privilege in a different app domain.
3. You can run different versions of DLL in every application domain.

In the next article, I am going to discuss the [Differences Between Strong and Weak Assembly in .NET Framework](#) with Examples. Here, in this article, I try to explain the **App Domain in .NET Framework** with Examples and I hope you enjoy this App Domain in .NET Application article.